OnboardAI: Team Study Guide for the Final Review

This is for every team member, written for someone who has not followed the project. Read it top to bottom once (about 40 minutes), then reread your own section in Part 4 until you can say it without notes.

Reading order for tonight:

1. **1 Team Study Guide** (this document), all of it.
2. **2 Final Review Demo**: the exact steps of the demonstration and who does what.
3. **3 Project Guide**, Part A (plain language). Part B only if you want depth.
4. **4 Final Evaluation**: every test case and result, if an examiner asks for detail.
5. **5 README**: how to install and run the project, if you want to try it yourself.

Part 1: The Project in Two Minutes

What it is

OnboardAI is a website that helps a company bring new employees on board. It does three jobs that HR staff normally do by hand:

- **Checks documents.** A new hire uploads a passport, offer letter, bank form or certificate. The system reads it, decides what kind of document it is, checks names, dates and signatures, and either approves it, rejects it with a reason, or sends it to a human in HR.
- **Answers questions.** A chat assistant answers questions like "how much leave do I get?" from the company's own handbook, quoting the section. If the handbook does not say, it admits it and offers to ask HR.
- **Predicts who is falling behind.** It estimates how likely each new hire is to get stuck, and explains why.

The demo uses two pretend companies with different rules: **Nimbus Labs** (a fintech start-up, 24 days of leave) and **Veltrix Corp** (a factory, 18 days of leave). Neither company can see the other's documents.

The one sentence to remember

"We built it, trained it on data we generated, then tested it on cases we wrote by hand, and we show both where it works and where it doesn't."

The honest headline

- Document checking: **16 out of 16** hand-written test documents correct, and **0** bad documents approved.
- The chatbot's message reader: **94%** on generated test messages but **73%** on hand-written ones. It learned the vocabulary of our templates. We say this openly; examiners respect it.

Part 2: Words You Will Hear

Term	What it means in plain words
Machine learning model	A program that learns a pattern from examples instead of being given rules.
Classifier	A model that puts things into categories, e.g. "this is a passport".

Term	What it means in plain words
Training data / test data	Examples the model learns from / examples kept aside to check it.
Synthetic (generated) data	Data our own programs created, because real passports and bank details cannot be collected.
Held-out / hand-written test set	48 messages, 16 questions and 16 documents we wrote ourselves after training, to test on unfamiliar wording.
Stratified split	Splitting data 75/25 so each category has the same share in both parts.
Data leakage	When test examples sneak into training, making scores look better than they are.
Cross validation (5-fold)	Train and test five times on different slices and average, so one lucky split cannot fool us.
TF-IDF	Turns text into numbers: a word scores high if it is frequent in this text but rare overall.
N-grams	Sequences of words ("annual leave") or letters ("lea", "eav"). Letter n-grams survive spelling damage.
Confidence	How sure the model is, from 0 to 100%. Low confidence means a human decides.
Accuracy	Share of answers that are correct.
Precision / recall / F1	Of what it flagged, how much was right / of what it should have flagged, how much it found / a balance of both.
ROC-AUC	For the risk model: the chance it ranks a person who stalls above one who doesn't. 0.5 is guessing, 1.0 is perfect. Ours is 0.81.
Calibration	Whether "70% risk" really means 7 in 10 such people stall. Ours is well calibrated.
Logistic regression / linear SVM	Simple, fast, explainable model types. We chose them on purpose over complicated ones.
LLM (large language model)	The AI that writes the chat replies. Ours is Qwen 2.5 (7B) running on the laptop through Ollama.
Agent	An LLM given a job description and a few tools it may use, e.g. "search the handbook".
Specialist agents	Our six agents, each owning one topic: onboarding, documents, IT, knowledge transfer, policy, escalation.
Routing	Deciding which specialist gets a message. Done by our small trained classifier, not the LLM.
Handoff / transfer	A specialist passing a message it cannot handle to the right one. Allowed once per message.
RAG (retrieval)	Search the handbook first, then let the LLM answer only from what was found. Stops it inventing policy.
Citation	The reply names the handbook and section it used, e.g. "Leave Policy (Nimbus Labs), Annual leave".
PII redaction	Removing personal data (passport, phone, bank numbers) before the message reaches the LLM.
OCR	Reading text from an image. Not installed on our laptop, so photos go to HR instead.
Escalation	Flagging a message to a human in HR, e.g. when the employee is very upset.

Term	What it means in plain words
Documentation gap	A question the handbook could not answer, shown to HR so they can write that page.

Part 3: How It Works, Step by Step

A document's journey

Example: Shreya uploads her passport against the requirement "Govt ID / Passport".

1. **Read.** The text is pulled out of the PDF.
2. **Classify.** The document classifier says "government ID, 98% confident".
3. **Extract.** Simple rules find the name ("Shreya Nair"), the expiry date, and whether there is a signature.
4. **Check.** Is it readable? Is it the type that was asked for? Is it expired, or expiring within 30 days? Does the name match the employee? Is it signed, if it must be?
5. **Decide.** Any serious failure (wrong document, expired, unsigned) means **rejected** with the reason. Anything uncertain (name mismatch, expiring soon, blank signature line, low confidence, unreadable photo) goes to **HR review**. Only a confident classification with every check passing is **approved**.

Why so cautious? Wrongly approving a bad document is the expensive mistake. Asking HR to look costs a minute.

HR can overrule any decision, and every overrule is recorded so it can be used to retrain the model later.

A chat message's journey

Example: Shreya types "How many days of annual leave do I get?"

1. **Classify.** In about 10 milliseconds, the intent classifier says "policy question, 99%" and the sentiment classifier says "neutral". No LLM is used yet.
2. **Protect.** Any personal numbers in the message are replaced with placeholders.
3. **Route.** Confidence is above 45%, so the message goes to the **Policy Agent**. (Below 45% it would go to the general Onboarding Agent.)
4. **Search.** The Policy Agent searches only Nimbus Labs' handbook and finds "Leave Policy, Annual leave: 24 days".
5. **Answer.** The LLM writes the reply and cites the section. If the best match had been weak, it would have said it has no documented answer and offered HR.
6. **Escalate if needed.** A very negative message (sentiment -0.60 or lower), or a complaint, is flagged to HR automatically.

If a message reaches the wrong specialist, that specialist can transfer it once to the right one, which must then answer.

The risk score

For each new hire the risk model looks at 12 numbers: progress, tasks done, days since last activity, documents rejected or pending, how positive their messages sound, and so on. It gives a probability of stalling and names the biggest reasons. Message sentiment turned out to be the strongest signal.

Part 4: Your Own Brief

Kritika Sah, Team Lead

You own: the problem, the objectives, the plan, and the story. You open, introduce the team, show the HR view, and close.

Opening (about 1 minute), say something like:

"Onboarding a new employee means a pile of documents to check and a stream of repetitive questions, all handled by hand. We built OnboardAI to automate the routine part safely: it verifies documents, answers policy questions from the company's own handbook, and flags new hires who are falling behind. Rohith built the application, Prisha evaluated it on test cases we wrote by hand, and Srikar built the risk model and the data. We will show it working, then show honestly how well it works."

Your demo part (1c): log in as HR (`hr1` / `@hr1`), open Shreya, show her uploads, the automatic verdicts and the activity log. Say: "HR can overrule any verdict, and every overrule is saved as a training label."

Closing: "Document checking was right on all 16 hand-written cases and never approved a bad document. The chat assistant answers correctly from the handbook, but our message classifier drops from 94 to 73 per cent on natural wording because it was trained on generated data. Training on real messages is the next step."

Questions likely to come to you:

- *Why generated data?* Real passports and bank records cannot lawfully be collected for a student project, and real outcomes do not exist because the system has never been deployed.
- *What was hardest?* Making the generated data hard enough to be meaningful; our first document classifier scored a perfect 1.000, which told us the data was too easy, so we rebuilt it.
- *What changed since mid term?* Models trained and evaluated, the full application, a hand-written evaluation that exposed five bugs (all fixed), and the team restructured into a lead and three developers.
- *What would you do next?* Real messages for training, a public document dataset with OCR, and retraining on HR's overrule decisions.

Rohith K Nair, Developer (application code)

You own: all the code: document pipeline, chat assistant, agents, policy search, server and website. You run the live demo in Part 1.

Your demo parts: 1a (upload documents as Shreya) and 1b (chat questions). Steps are in the demo PDF.

Know cold:

- Stack: React + Vite front end, FastAPI + SQLite back end, scikit-learn models, Qwen 2.5 7B via Ollama for chat, LangChain agents.
- Document decision rules (Part 3 above) and thresholds: approve needs confidence 0.80 or more; expiry warning at 30 days.
- Routing threshold 0.45; escalation at sentiment -0.60; policy "no answer" below 0.42.
- Six specialists, each with only its own tools; tools are locked to the signed-in employee.

Questions likely to come to you:

- *Why not let the LLM pick the specialist?* The classifier decides in milliseconds instead of seconds, gives the same answer every time, costs nothing, and we can measure its accuracy.
- *How do you stop the LLM inventing policy?* It must search before answering and cite the section; weak search results tell it to decline.
- *Could someone ask about another employee?* No: tools take the employee id from the login, never from the message.
- *Why a local model?* No cost, no API key, and employee messages never leave the machine. Switching to Gemini or Groq is one setting.

- *Why does a reply take 30 seconds?* A 7-billion-parameter model on a laptop GPU. Everything before the LLM takes milliseconds.

Prisha Saini, Developer (evaluation and tests)

You own: the hand-written test cases, the Evaluation Lab, the results, and the automated test suite. You present Part 2 of the demo.

What you built the evaluation to answer: "Does the system work on inputs it has never seen, written the way people actually write?"

The test sets (all written by hand after training):

- 48 chat messages, 6 for each of 8 intents, with expected intent and sentiment.
- 16 policy questions: 12 the handbook answers, 2 it does not, 2 that only the other company's handbook answers.
- 16 documents with the verdict an HR officer would give. Dates are calculated from today, so "expires in 10 days" is always true.

The numbers to know cold:

Test	Result
Intent, hand-written / generated	73% / 94%
Sentiment, hand-written	67%
Policy search, right section first / in top 3	83% / 92%
Policy search, declines / no cross-company leaks	2 of 2 / 2 of 2
Full answers: correct fact / cited / declined when it should	11 of 12 / 9 of 12 / 3 of 4
Documents correct / bad ones approved	16 of 16 / 0
Automated tests	62, all passing, about 20 seconds

Questions likely to come to you:

- *Why is accuracy lower on your own test set?* The model was trained on template messages; natural phrasing is different. We built the hand-written set to measure exactly that gap.
- *Did you tune the system on your test set?* We fixed five bugs the test set exposed but retrained no model on it, and we show the remaining failures. Since the team wrote both, an outside test set would be stronger.
- *Why do some answers count as fails when the fact is right?* Our rule is strict: a policy answer must also cite the handbook section. Three correct answers did not cite, so they fail.
- *Why no language model in the automated tests?* LLM replies vary from run to run, so they cannot be a reliable pass/fail gate. They are evaluated separately in Layer 3.
- *What is the most important number?* Zero bad documents approved: that is the mistake that costs a company.

Pasham Srikar, Developer (risk model and data)

You own: the three datasets, preparing and splitting them, and the risk prediction model. You present Part 2c.

The data:

Dataset	Rows	What it is
Documents	4,950	9 types, 550 each, damaged in 3 quality tiers
Messages	3,360	8 intents and 3 sentiments
Journeys	6,000	12 numbers per simulated employee, outcome: stalled or not (43% stalled)

Split 75/25, stratified, `random_state=42`. No missing values. Outliers kept on purpose: a long-inactive employee is exactly who we want to catch.

The risk model: logistic regression, chosen over gradient boosting because it scored higher (ROC-AUC 0.795 vs 0.777), is better calibrated, and explains itself. Cross validated ROC-AUC **0.81**. When it says 88% risk, 88% of such people really stall. The strongest signal is average message sentiment.

Your demo part (2c): on the HR dashboard show "Predicted stall risk" with the main reason under each person; log in as `hr2` / `@hr2` to open Adithya (blocked by an expired safety certificate) and show his risk drivers; point to "Documentation gaps".

Questions likely to come to you:

- *How did you make generated journeys realistic?* Each simulated person has hidden traits (engagement, friction, support) that drive the visible numbers with noise, plus a hidden factor the model can never see, so a perfect score is impossible by design.
- *Why ROC-AUC and not accuracy?* The risk score ranks people for HR's attention; ROC-AUC measures ranking quality, and calibration checks the probabilities mean what they say.
- *What was the leakage issue?* 70 duplicate messages ended up in both training and test. We measured it: the duplicates scored lower, not higher, so they did not inflate results. It is documented.
- *Why did removing features barely change the score?* Three progress measures correlate above 0.90; removing one leaves the same information in the others.

Part 5: Questions Anyone Might Get

- **Is this ready for real use?** No. It needs real training data, OCR, real logins and a security review. It shows the approach works and where it fails.
- **What is AI and what is rules?** Classifying documents and messages, the risk score, and writing chat replies are learned models. Date and signature extraction and the approve/reject policy are explicit rules, on purpose, so decisions are explainable.
- **Can it detect forgery?** No. It checks type, name, expiry and signature. HR can overrule anything.
- **Why does the chat take so long?** The language model runs on the laptop. Everything else is instant.
- **What if the LLM is offline?** The dashboard and document checks still work; the chat says AI is offline.

Things not to say

- Do not say the system is "accurate" without saying on what: generated data or hand-written data.
- Do not present the 99.6% sentiment figure as real ability; it reflects how easy our templates are.
- Do not claim it detects forged documents.
- If you do not know an answer, say "Rohith built that part, he can answer", rather than guessing.

Part 6: Logins and Files

Login	Password	Who
employee 1	@1	Shreya, Nimbus Labs
employee 2	@2	Adithya, Veltrix Corp
hr1	@hr1	HR for Nimbus Labs
hr2	@hr2	HR for Veltrix Corp

What	Where
Test documents to upload	`backend/demo_documents/final_review/`
Evaluation Lab	Log in as HR, click Evaluation Lab in the header
Hand-written test cases	`backend/evaluation/chat_cases.py`, `doc_cases.py`
All results, offline	`docs/FINAL_EVALUATION.md` and its PDF
Model reports	`backend/ml/artifacts/*_report.md`